# mdtopdf デバッグ用サンプル

このファイルは、Markdown から PDF への変換結果を確認するためのサンプルです。 見出し、装飾、リスト、コード、表、改ページなど、崩れやすい要素を一通り入れています。

---

## 1. 見出しレベル

# 見出し1

## 見出し2

### 見出し3

#### 見出し4

##### 見出し5

###### 見出し6

## 2. インライン装飾

通常のテキストに、**太字**、*斜体*、***太字かつ斜体***、~~取り消し線~~、`インラインコード` を混ぜた文章です。

リンクの例: Anthropic と、URL をそのまま書いた https://example.com です。

日本語と English が混在する文章でも、フォントが正しく適用されるか確認します。漢字、ひらがな、カタカナ、半角ｶﾀｶﾅ、全角英数ＡＢＣ１２３、記号「」『』（）【】〜…！？ も含めています。

絵文字の表示確認: ✅ ⚠️ 🚀 📄

## 3. リスト

### 箇条書き

- りんご
- みかん
  - 温州みかん
  - デコポン
    - さらに深い階層
- ぶどう

### 番号付きリスト

1. 最初の手順
2. 次の手順
   1. 細かい手順 A
   2. 細かい手順 B
3. 最後の手順

### タスクリスト

- [x] 読み込み処理を実装
- [x] HTML への変換を実装
- [ ] PDF 出力を実装
- [ ] CSS テーマを追加

## 4. 引用

> これは引用ブロックです。 複数行にわたる引用も確認します。
>
> > ネストした引用です。

## 5. コードブロック

### TypeScript

```ts
import { Command } from 'commander';
import fs from 'node:fs';

interface Options {
```

```
  output?: string;
}

const program = new Command();

program
  .name('mdtopdf')
  .argument('<file>', 'input Markdown file')
  .option('-o, --output <path>', 'output PDF path')
  .action(async (file: string, options: Options) => {
    const markdown = fs.readFileSync(file, 'utf-8');
    console.log(`read ${markdown.length} chars`, options.output ?? '(default)');
  });

await program.parseAsync();
```

### シェル

```
npm install commander marked puppeteer
npx tsx src/index.ts sample.md -o sample.pdf
```

### 横に長い行（折り返し・はみ出しの確認）

```
ここにとても長い一行が入ります。ここにとても長い一行が入ります。ここにとても長い一行が入ります。ここにとても長い一行が入ります。ここにとても長い一行が入ります。ここにとても長い一行が入ります。
const veryLongVariableName = someFunction(argumentNumberOne, argumentNumberTwo, argumentNumberThree, argumentNumberFour, argumentNumberFive);
```

## 6. 表

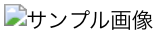
| コマンド | 説明 | 必須 |
|---|---|---|
| `<file>` | 入力する Markdown ファイル | ✅ |
| `-o, --output <path>` | 出力する PDF のパス | |
| `-h, --help` | ヘルプを表示 | |
| `-V, --version` | バージョンを表示 | |

### 列の揃えと長い文章を含む表

| 左揃え | 中央揃え | 右揃え |
|---|:---:|---:|
| left | center | right |
| 長めの文章を入れて、セル内で折り返されるかを確認します。長めの文章を入れて、セル内で折り返されるかを確認します。 | 中央 | 12,345 |
| short | text | 6 |

## 7. 画像

次の画像は相対パスで参照しています。`images/sample.png` を用意すると、相対パスの解決を確認できます。 ファイルがない場合は、画像が表示されない（壊れた画像になる）ことの確認になります。



## 8. 水平線

上の文章。

---

下の文章。

## 9. 改ページの確認用セクション

ここから先は、複数ページにまたがる場合の挙動を確認するための長めの文章です。 見出しがページの最後に取り残されないか、コードブロックや表がページの途中で分断されないかを見てください。

### 9-1. 長い段落

吾輩はサンプルである。名前はまだない。どこで生れたかとんと見当がつかぬ。何でも薄暗いじめじめした所でテキストを書いていたことだけは記憶している。この文章は、ページ送りの確認のために、意味のない内容を繰り返しています。この文章は、ページ送りの確認のために、意味のない内容を繰り返しています。この文章は、ページ送りの確認のために、意味のない内容を繰り返しています。この文章は、ページ送りの確認のために、意味のない内容を繰り返しています。この文章は、ページ送りの確認のために、意味のない内容を繰り返しています。

The quick brown fox jumps over the lazy dog. This paragraph is here only to fill space so that the page break behavior can be checked. The quick brown fox jumps over the lazy dog. This paragraph is here only to fill space so that the page break behavior can be checked. The quick brown fox jumps over the lazy dog.

### 9-2. 長いリスト

1. 項目 1
2. 項目 2
3. 項目 3
4. 項目 4
5. 項目 5
6. 項目 6

7. 項目 7
8. 項目 8
9. 項目 9
10. 項目 10
11. 項目 11
12. 項目 12
13. 項目 13
14. 項目 14
15. 項目 15
16. 項目 16
17. 項目 17
18. 項目 18
19. 項目 19
20. 項目 20

### 9-3. 長いコードブロック

```
function fizzBuzz(n: number): string[] {
  const result: string[] = [];
  for (let i = 1; i <= n; i++) {
    if (i % 15 === 0) {
      result.push('FizzBuzz');
    } else if (i % 3 === 0) {
      result.push('Fizz');
    } else if (i % 5 === 0) {
      result.push('Buzz');
    } else {
      result.push(String(i));
    }
  }
  return result;
}

function sum(numbers: number[]): number {
  return numbers.reduce((acc, cur) => acc + cur, 0);
}

function average(numbers: number[]): number {
  if (numbers.length === 0) return 0;
  return sum(numbers) / numbers.length;
}

console.log(fizzBuzz(30).join('\n'));
console.log(average([1, 2, 3, 4, 5]));
```

### 9-4. 長い表

| No. | 名前 | 値 |
|---|---|---|
| 1 | alpha | 100 |
| 2 | bravo | 200 |
| 3 | charlie | 300 |
| 4 | delta | 400 |
| 5 | echo | 500 |
| 6 | foxtrot | 600 |
| 7 | golf | 700 |
| 8 | hotel | 800 |
| 9 | india | 900 |
| 10 | juliet | 1000 |
| 11 | kilo | 1100 |
| 12 | lima | 1200 |
| 13 | mike | 1300 |
| 14 | november | 1400 |
| 15 | oscar | 1500 |

## 10. 特殊文字のエスケープ

HTML として解釈されやすい文字の確認です: `<div>`、`&`、`"引用符"`、`'アポストロフィ'`、`Ctrl + C`

バックスラッシュでのエスケープ: *アスタリスク*、# シャープ、_アンダースコア_

---

以上です。最後まで崩れずに出力されていれば、基本的な変換は問題ありません。